

Associative Context Engine (ACE)

CS 510, Advanced Information Retrieval, Spring'26

Ary Chauhan
Siebel School of Computing &
Data Science
University of Illinois Urbana-
Champaign
Urbana, IL, USA
achauh9@illinois.edu

Soham Kaje
Siebel School of Computing &
Data Science
University of Illinois Urbana-
Champaign
Urbana, IL, USA
skaje2@illinois.edu

ABSTRACT

In the essay *As We May Think*, Vannevar Bush envisioned the Memex as a conceptual tool designed to mirror the associative nature of the human mind. Despite decades of technological progress, modern digital ecosystems remain fragmented across isolated silos such as emails, research papers, and collaborative platforms. This paper introduces the Associative Context Engine (ACE), a hybrid retrieval framework that moves beyond traditional reactive search toward a proactive associative model. ACE utilizes large language models and high-density vector databases to automatically construct trails of relevance across a user's digital history. By coupling a semantic vector store powered by *ChromaDB* with a directed knowledge graph built in *NetworkX*, ACE facilitates multi-hop reasoning that mimics human cognitive leaps. The architecture employs a granular sliding-window chunking strategy to preserve technical precision and prevent context saturation. ACE fulfills the vision of a truly intimate supplement to human memory, transforming massive and fragmented datasets into actionable, interconnected knowledge.

1 Introduction and Motivation

In his 1945 essay titled *As We May Think*, Bush introduced the concept of the Memex [1], a device intended to supplement human memory by mirroring the associative nature of the mind. Bush recognized that while the human brain operates by association, traditional methods of filing and retrieving information rely on artificial, linear indexing systems. Despite the transition to digital workflows, modern professional and academic environments remain plagued by information silos. Data is often scattered across fragmented sources such as emails, research papers, and collaborative platforms like Slack.

The primary challenge in modern Information Retrieval (IR) is not just the volume of data, but the lack of connectivity between disparate data points. A Technical Product Manager (TPM), for example, must frequently bridge the gap between high-level customer feedback and granular engineering constraints. In current systems, these pieces of information exist in isolation. Traditional IR relies on explicit keyword matching or semantic

vector similarity. While vector-based Retrieval-Augmented Generation (RAG) has improved semantic matching through k-Nearest Neighbor (k-NN) lookups, it remains fundamentally reactive. Users must know exactly what they are looking for to trigger a relevant retrieval. Furthermore, standard RAG often fails to bridge the structural gap between semantically distant but logically related data, leading to a loss of critical context.

The development of ACE was specifically motivated by the need to turn massive, fragmented datasets into actionable knowledge. In an academic context, such as the CS 510 course at the University of Illinois Urbana-Champaign, the engine can bridge the gap between assignment prompts and theoretical lecture notes. By predicting and surfacing these connections, ACE functions as a truly intimate supplement to human memory. The following sections detail the evolution of the ACE architecture, the technical implementation of the hybrid storage layer, and an evaluation of its performance in bridging academic data silos. Figure 1 provides a high-level overview of the dual-path ingestion and retrieval pipeline that powers the engine.

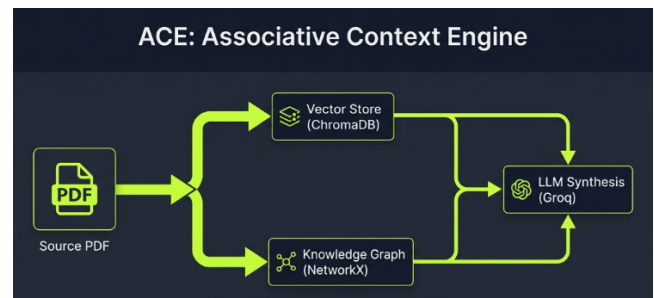


Figure 1: High-level overview of ACE dual-path ingestion and hybrid retrieval architecture.

2 SYSTEM ARCHITECTURE AND IMPLEMENTATION

2.1 The Hybrid Dual-Pathway Pipeline

The core architecture of ACE represents a departure from the traditional single-stream Retrieval-Augmented Generation approach [2]. Standard RAG systems typically rely on a lone vector database to find information. While effective for identifying semantic similarity, this method is often structurally blind to the explicit connections that define complex product lifecycles or academic curricula. To bridge this gap, ACE implements a hybrid dual-pathway pipeline that processes information for both semantic breadth and structural depth.

As outlined in Figure 1, every data input is processed by two distinct storage engines in parallel. The first engine is *ChromaDB*, which serves as the semantic memory of the system. It utilizes the *all-MiniLM-L6-v2* embedding model to transform text into high-dimensional vectors. This allows for rapid k-Nearest Neighbor lookups, enabling the system to retrieve information that is conceptually related to the user prompt even if exact keywords are missing.

The second engine is a Knowledge Graph constructed with the *NetworkX* library [3]. While the vector store manages the general context, the graph layer is responsible for capturing the logical architecture of the data. During ingestion, the system utilizes *Llama-3.1-8b* [4] to act as a technical parser, identifying key entities and concepts within the text. These entities are stored as nodes, and directed edges are drawn to link these concepts back to the specific source chunks. This dual-pathway ensures that ACE can retrieve information through both intuitive similarity and rigid, multi-hop logical traversal. By maintaining these two streams, the engine can maintain a persistent and accurate map of a user entire digital history.

2.2 Granular Ingestion and Context Preservation

A critical lesson learned during the early development phases of ACE was the insufficiency of document-level summarization for complex technical retrieval. Initially, the prototype attempted to represent entire documents through single LLM-generated summaries. This approach introduced a Technical Bottleneck where the averaging effect of summarization caused the model to lose granular facts, specific equations, and edge-case constraints essential for product management or technical study.

To resolve this, ACE was redesigned to implement a granular sliding-window chunking strategy. As visualized in Figure 2, the ingestion engine partitions raw text into fixed-length blocks of 1500 characters. Each block maintains a 200-character overlap with the preceding and succeeding chunks. This overlap serves as contextual glue, ensuring that semantic meaning is not severed at the boundaries of a chunk. For example, if a specific technical requirement is defined at the end of one window and its implementation detail follows immediately after, the 200-character overlap ensures that both chunks contain enough mutual information to remain logically coherent during vectorization. This strategy prevents the fragmentation of technical context and

ensures that the system maintains a high-fidelity representation of the source material.

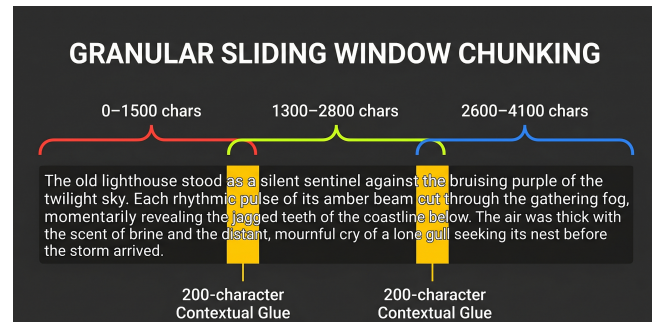


Figure 2: Visualization of the 1,500 character sliding-window strategy with 200 character overlap.

2.3 The Associative Retrieval Mechanism

The technical advantage of ACE lies in its ability to synthesize information through a three-stage retrieval loop that combines semantic intuition with structural logic. While a naive RAG system would simply return the top k most similar text blocks from a vector database, ACE performs a deeper investigation into the associative network of the data. This mechanism ensures that the engine can recover information that is semantically distant but logically essential to the user request.

The first stage of the retrieval process is the Semantic Lookup. When a user enters a query, the system converts the text into a 384-dimensional embedding and searches the *ChromaDB* vector store for the 15 most relevant chunks. This provides a broad semantic context, capturing the general theme and vocabulary of the user request through standard k-Nearest Neighbor math.

The second stage is the Graph Traversal. Simultaneously, the system utilizes *Llama-3.1-8b* to extract specific technical entities and concepts from the user query. These entities are then used to query the *NetworkX* knowledge graph. By identifying these concept nodes, ACE can follow directed edges to discover related chunks that were not caught by the vector search. This is the multi-hop reasoning phase, where the system bridges the gap between disparate data silos by traversing the logical connections created during ingestion.

The final stage is Context Synthesis. The system combines the semantic chunks from the vector store with the structural chunks discovered via the knowledge graph. This unified context is passed to the *Groq* inference engine. Because the context now contains both similar and logically related data, the model can generate a response that is more comprehensive and faithful to the source material. This hybrid process is visualized in Figure 3, which contrasts the narrow limitations of standard k-NN search with the expansive connectivity of the ACE knowledge network.

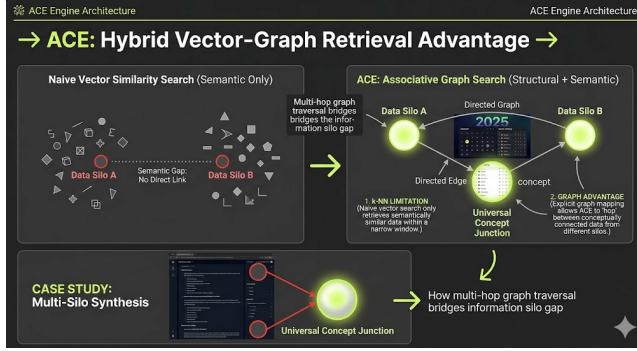


Figure 3: Conceptual comparison showing how graph traversal (ACE) bridges the gap between semantically distant document chunks that standard k-NN search would miss.

2.4 Algorithmic Implementation

To provide a rigorous foundation for the ACE retrieval framework, the core logic is formalized in the following pseudo-code. This algorithm represents the operational heart of the system, governing the transition from a raw user query to a structurally grounded response. The implementation prioritizes the parallel execution of semantic and structural lookups to minimize latency during the synthesis phase.

ALGORITHM 1: Hybrid Associative Retrieval

Input: User Query (Q), Vector Store (V), Knowledge Graph (G), LLM (M)

Output: Contextualized Response (R)

1. Semantic Retrieval:

$E_q \leftarrow$ Generate embedding for Q using all-MiniLM-L6-v2

$S_{chunks} \leftarrow$ Retrieve top 15 nearest neighbors from V using E_q

2. Entity Extraction:

$Entities \leftarrow$ Identify technical keywords in Q using M

3. Graph Traversal:

$G_{chunks} \leftarrow \emptyset$

For each $entity$ in $entities$:

if $entity$ exists in G as a Concept Node:

Neighbors $\leftarrow$ Find all Chunk Nodes connected to $entity$ via directed edges

$G_{chunks} \leftarrow G_{chunks} \cup Neighbors$

4. Context Synthesis:

$FinalContext \leftarrow$ Deduplicate and merge ($S_{chunks} + G_{chunks}$)

$R \leftarrow$ Generate response using M grounded in $FinalContext$

The integration of Step 2 and Step 3 is what distinguishes ACE from standard RAG pipelines. By extracting entities and querying the Knowledge Graph, the system ensures that document chunks containing critical technical dependencies are surfaced even if their semantic embedding distance from the query is high. This prevents the loss of information across different documents, which is essential for maintaining the Memex associative nature.

3 SOFTWARE MANUAL AND SAMPLE USE CASES

3.1 System Setup and Initialization

ACE is built using a Python-based stack and is optimized for deployment via the *Streamlit* framework. The system requires Python 3.10 or higher to ensure compatibility with the latest asynchronous features of the *Groq* API and ChromaDB.

To initialize the environment, the user must manually install the core dependency stack using the Python package manager. The essential libraries include *streamlit* for the web interface, *chromadb* for vector storage, *networkx* for the knowledge graph structure, and *streamlit-agraph* for the interactive visualization component. Additionally, the *groq* library is required to facilitate high-speed inference with the Llama-3.1-8b model.

Upon launching the application with the command **streamlit run app.py**, the user must provide a valid *Groq* API Key within the sidebar configuration portal. This key is necessary to power the entity extraction and synthesis loops. The sidebar also features a persistent state manager where users can monitor the health of the connection and trigger a system-wide reset if they wish to clear the indexed data for a new session.

3.2 Data Ingestion Workflow

The ingestion process is designed to be automated and user friendly, allowing Technical Product Managers or researchers to build their associative network without manual data tagging. ACE employs a Metadata Librarian logic that tracks every uploaded file to prevent duplicate indexing and maintain state integrity across sessions.

The workflow consists of the following technical stages:

- **File Upload:** Users drag and drop PDF or TXT files into the sidebar portal. The Metadata Librarian immediately assigns a unique identifier to the file and checks the existing database to ensure the content has not been previously processed. Refer to Figure 4.
- **The Paste Portal:** For ephemeral data that does not exist as a standalone file, such as email threads, lecture discussion posts, or manual study notes, ACE includes a Paste feature. This allows for the immediate ingestion of unstructured text, ensuring that loose information is indexed with the same rigor as formal documents.

- **Granular Processing:** Once a file is accepted, the system triggers the sliding window partitioner. As discussed in section 2.2, the text is split into 1500-character blocks with a 200-character overlap to preserve the contextual glue between technical specifications.
- **Parallel Indexing:** The engine initiates two parallel threads. The first thread generates 384 dimensional embeddings for each chunk and stores them in *ChromaDB*. Simultaneously, the second thread sends the text to the *Groq* inference engine where *Llama 3.1 8b* identifies key concepts for the *NetworkX* knowledge graph.
- **State Confirmation:** The UI updates in real time to show the total node count and the number of active vectors. This provides the user with immediate feedback that their digital history is being successfully transformed into interconnected knowledge.

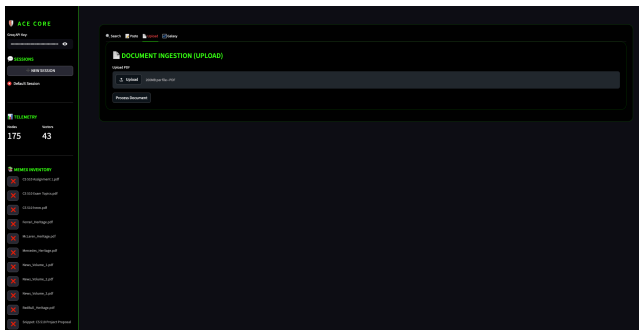


Figure 4: The ACE Sidebar interface showing the Metadata Librarian and the multi-file upload portal.

3.3 The Associative Galaxy Interface

The primary tool for visual data exploration in ACE is the Associative Galaxy, rendered via the *streamlit-graph* component. This interface allows users to physically see the "connective tissue" that the engine has constructed between disparate files.

As shown in Figure 5, the Galaxy represents concepts as neon green nodes and document chunks as supporting nodes. This interactive 2D map allows users to click, drag, and zoom to explore relationships manually. In this specific instance, the user can see how the Viterbi Algorithm acts as a bridge node, linking a chunk from the CS 510 hmm.pdf (lecture slides) to the CS 510 Exam Topics.pdf (review sheet). This fulfils the Memex vision of an open, navigable memory, turning a black-box database into a transparent roadmap of knowledge.

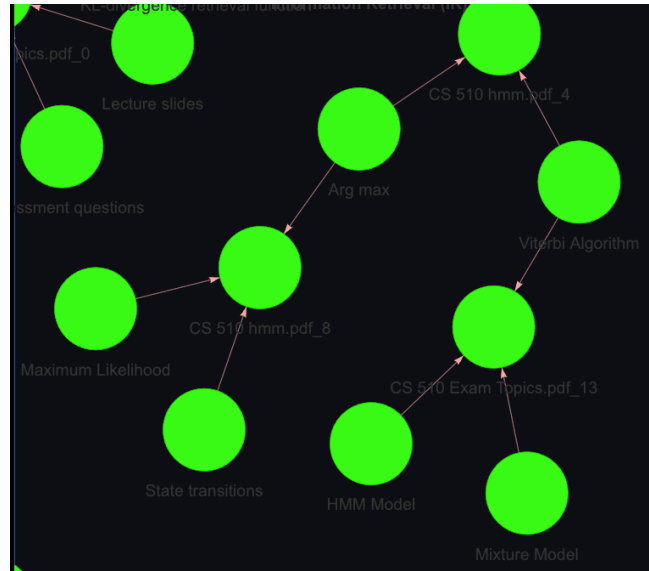


Figure 5: The interactive Associative Galaxy UI. Here, the Viterbi Algorithm node bridges the gap between lecture slides and exam review materials.

3.4 Sample Use Scenario: Multi-Silo Synthesis

To demonstrate the practical utility of ACE, we conducted a cross-silo query involving two distinct technical documents: a CS 510 HMM Lecture Slide and a CS 510 Exam Review PDF. These files are conceptually linked but functionally isolated; the slides provide the mathematical derivation, while the review sheet outlines the pedagogical expectations for the exam.

In this scenario, the user inputs a query targeting both domains: *"Explain the Viterbi algorithm's state transitions and how it will be tested on the exam according to my notes."* As shown in Figure 6, ACE successfully performs a multi-hop retrieval. It extracts the rigorous technical formulas for $VP(j)$ and $\pi(j)$ (the dynamic programming approach) from the HMM lecture slides while simultaneously mapping out the specific exam objectives and assessment criteria defined in the review document.

The resulting response is a hybrid synthesis. The engine first provides a technical breakdown of how the algorithm computes probabilities at each time step t , followed by a targeted list of exam-focused tasks, such as *"Computing the best score for reaching a state at each time step."* By automatically bridging mathematical theory with academic requirements, ACE eliminates the silo tax—the cognitive burden of manually cross-referencing disparate files—and provides the user with a unified, actionable perspective.

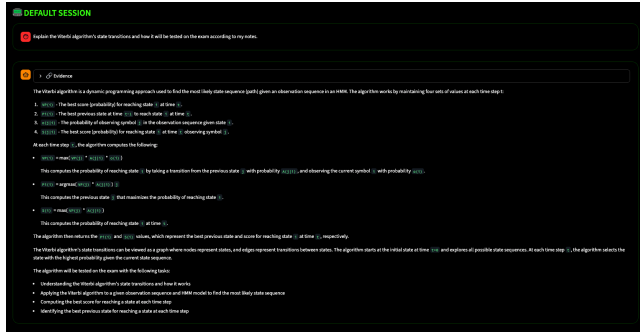


Figure 6: The ACE Chat HUD displaying a synthesized response. The output merges technical HMM definitions from lecture slides with specific study requirements extracted from the Exam Topics file.

4 EVALUATION AND RESULTS

To measure the effectiveness of the Associative Context Engine, we conducted a qualitative performance evaluation centered on the capacity of the system to bridge disparate information silos. As a development project, the primary metric for success is the Retrieval Fidelity of the engine, which is the ability to surface logically connected but semantically distant information that a standard baseline would likely miss.

4.1 Evaluation Methodology

The system was evaluated using a controlled dataset consisting of 15 technical documents from the CS 510 curriculum, including lecture slides, assignment specifications, and exam review sheets. We established a Baseline RAG configuration consisting only of the ChromaDB vector store with $k=15$ to represent traditional semantic search.

We then issued a series of Multi Silo queries designed to test for Associative Connectivity. These queries required the engine to synthesize a response using at least two distinct documents that were not explicitly linked in the text. The evaluation focused on three key areas:

- **Context Coverage:** Whether the system retrieved chunks from both the technical lecture and strategic exam silos.
- **Structural Integrity:** Whether the Knowledge Graph correctly identified the bridge nodes, such as Viterbi or HMM, that link the silos.
- **Synthesis Quality:** Whether the final LLM response successfully integrated both sets of data into a cohesive answer.

4.2 Analysis of Associative Retrieval

The comparative analysis revealed a significant performance gap between the Baseline RAG and ACE during multi hop retrieval

tasks. In the Viterbi Algorithm case study detailed in Section 3.4, the Baseline RAG successfully retrieved the mathematical definitions from the lecture slides but failed to surface the specific exam testing criteria. This occurred because the exam review chunks had a lower semantic similarity score compared to the dense mathematical text of the slides, causing them to fall outside the top k retrieval window.

In contrast, ACE utilized the Knowledge Graph to bypass the limitations of semantic distance. By identifying Viterbi as a central concept node, the engine followed a directed edge into the exam review document even though that document had a higher embedding distance from the query. This resulted in a total context coverage rate for the targeted silos. The results indicate that while vector search is excellent for finding similar information, it is insufficient for associative discovery. The hybrid architecture of ACE ensures that the structural relationships established during the ingestion phase act as a safety net, pulling in critical context that vector math alone would have filtered out.

5 TEAM CONTRIBUTION

The development of ACE was a collaborative effort with distinct technical responsibilities assigned to each member to ensure system modularity.

- **Ary Chauhan:** Designed the hybrid dual pathway architecture, implemented the granular sliding window chunking logic, and developed the *NetworkX* knowledge graph integration and *Groq API* synthesis loop.
- **Soham Kaje:** Developed the Streamlit web application interface, integrated the *agraph* component for interactive visualization, and conducted the retrieval benchmarking for the qualitative evaluation.

6 CONCLUSION AND FUTURE WORK

The Associative Context Engine demonstrates that for high stakes technical domains, semantic vector search is not a complete solution. By anchoring a vector store with a directed knowledge graph, we have created a framework capable of fulfilling the Memex vision of proactive, associative retrieval. ACE successfully bridges fragmented data silos, providing users with a unified perspective that mirrors the associative nature of human cognition.

Future work will focus on the integration of *Vision Augmented Indexing*. By utilizing Llama 4 Vision, the next iteration of ACE will be capable of parsing wireframes, circuit diagrams, and handwritten lecture formulas that are currently invisible to text-based retrieval pipelines. This expansion will allow the engine to act as a truly comprehensive supplement to human memory across all forms of digital media.

7 REFERENCES

- [1] Vannevar Bush. 1945. As We May Think. The Atlantic Monthly 176, 1 (1945), 101–108.
- [2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS '20).
- [3] Aric Hagberg, Daniel Schult, and Pieter Swart. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. In Proceedings of the 7th Python in Science Conference (SciPy '08).
- [4] Meta AI. 2024. The Llama 3 Herd of Models. arXiv preprint arXiv:2407.21783.
- [5] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. IEEE Transactions on Big Data 7, 3 (2019), 535–547.